

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
УЛЬЯНОВСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ

Факультет информационных систем и технологий
Кафедра «Измерительно-вычислительные комплексы»

**Отчет по лабораторной работе №1
по дисциплине «Системы искусственного интеллекта»**

Выполнил:
студент группы ИСТбд-31 Коняхин Н.С

Проверил:
Преподаватель Шишкин В.В.

Ульяновск, 2026

Цель:

Разработать алгоритм решающий задачу о разбиении городов на K кластеров и сравнить результат с работой ИИ решающего ту же задачу на тех же данных.

1.Алгоритмическое решение

- Суть алгоритма построена на создании матрицы смежности из строковых данных формата (название города , X,Y).
- Далее из находятся K кластеров по следующему принципу ,расчитывается средняя координата (центр всех точек) затем находятся точки которые имеют наибольшую разность координат с средней точкой, полученные точки и есть центроиды кластеров.
- После для каждого города итерируемся по строкам матрицы и определяем ближайший город центроид для этой точки , добавляем город в соответствующий кластер.

2.Решение с помощью ИИ

решение основано на методе K -средних. это популярный алгоритм обучения без учителя в машинном обучении, используемый для разбиения набора данных на заранее заданных кластеров (групп). Он итеративно минимизирует расстояния между точками и центрами кластеров (центроидами), группируя схожие объекты вместе.

- Сначала мы извлекаем данные из файла
- после создаем модель и обучаем ее
- далее передаем наши данные обученной модели и выводим полученный результат

3. Самописная реализация метода k-means

Суть работы метода следующая . С помощью метода локтя мы выбираем оптимальное количество K для метода к-средних. В самом методе ,случайным образом выбираются центроиды по заданному количеству K , после для каждого кластера находится средняя точка и выбирается наиболее близкая к ней точка из кластера. Поиск новых центроидов повторяется до тех пор пока набор центроидов не перестанет изменяться .

- Выбор случайных точек по числу K
- поиск средней точки для каждого кластера
- выбор ближайшей точки до средней
- проверка на изменения набора кластеров

4.Графики работы алгоритмов по времени результат работы на случайных данных:

Алгоритмическое решение

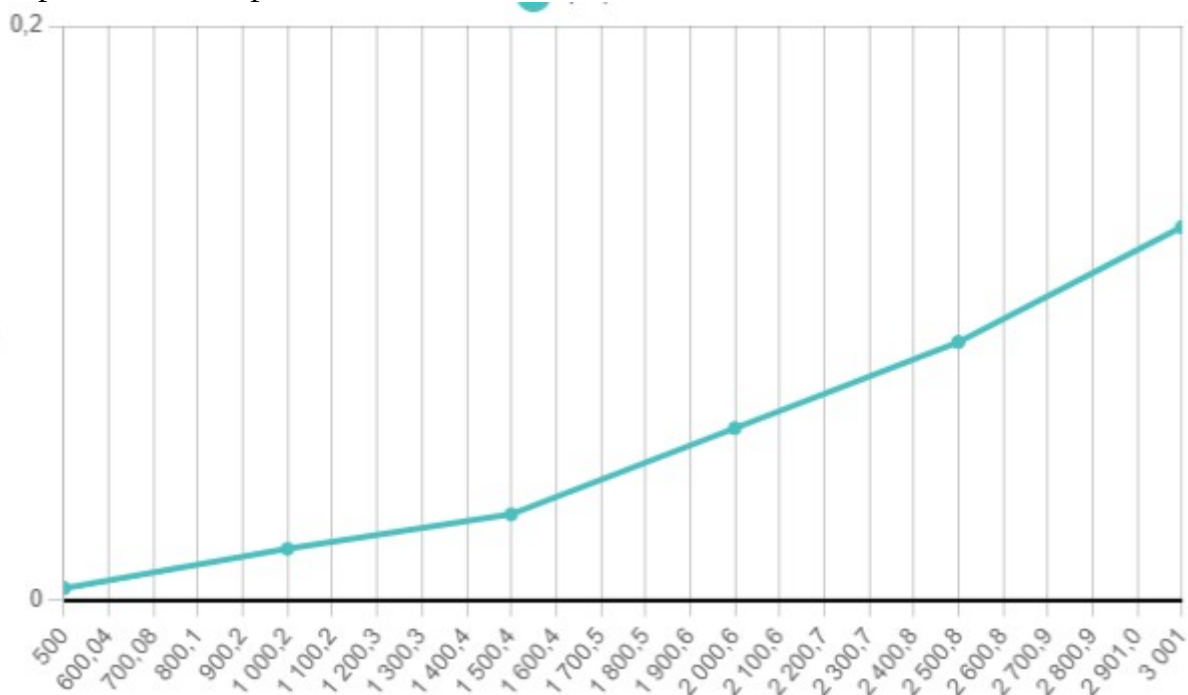


Таблица результатов для алгоритмического решения

№	x	y	Действия
1	500	0.0043	▲ ▼ Удалить
2	1000	0.018	▲ ▼ Удалить
3	1500	0.03	▲ ▼ Удалить
4	2000	0.06	▲ ▼ Удалить
5	2500	0.09	▲ ▼ Удалить
6	3000	0.13	▲ ▼ Удалить

График работы ИИ

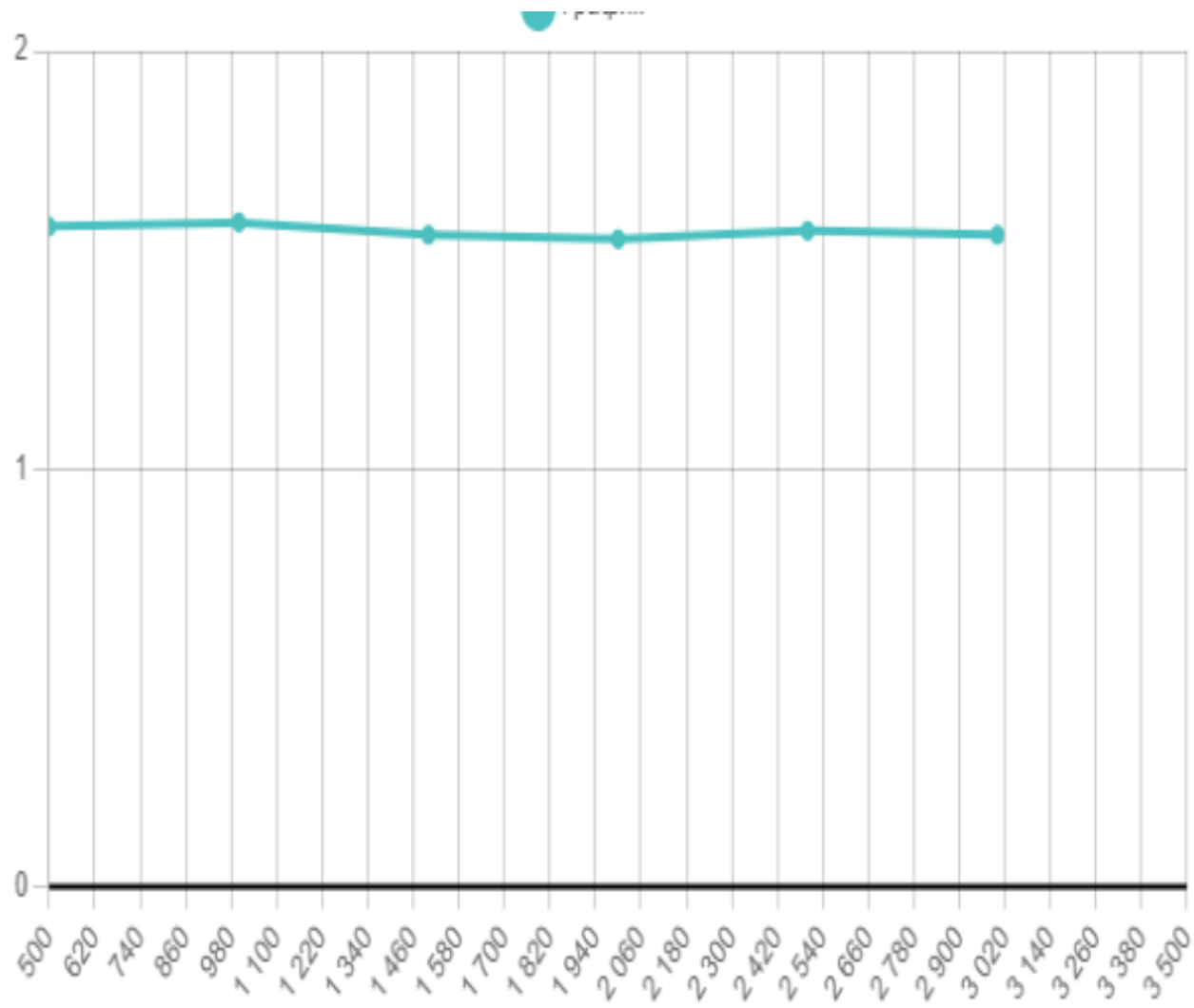
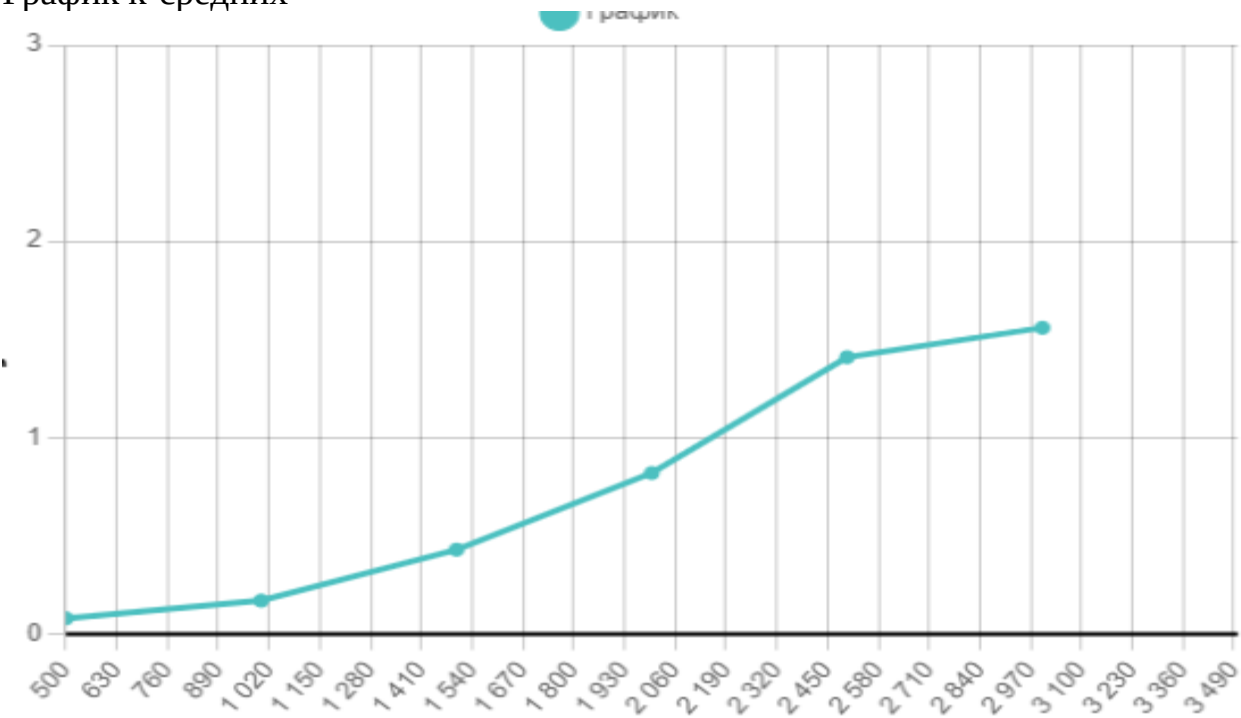


таблица результатов ИИ

№	x	y	Действия
1	500	1.58	▲▼Удалить
2	1000	1.59	▲▼Удалить
3	1500	1.56	▲▼Удалить
4	2000	1.55	▲▼Удалить
5	2500	1.57	▲▼Удалить
6	3000	1.56	▲▼Удалить

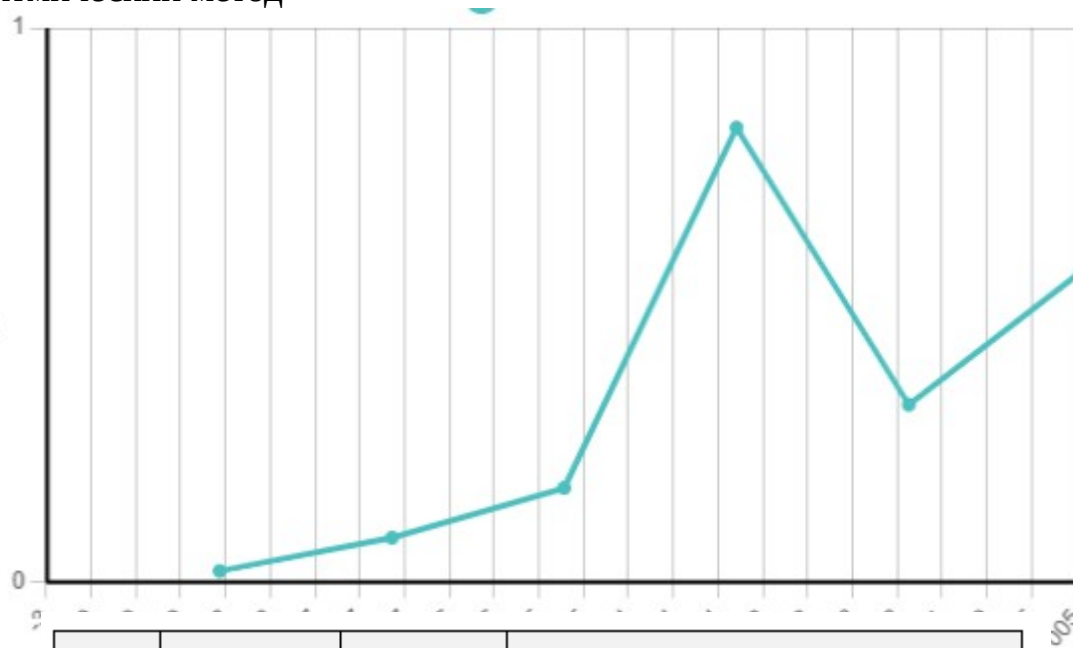
График к-средних



№	x	y	Действия
1	500	0.08	▲ ▼ Удалить
2	1000	0.17	▲ ▼ Удалить
3	1500	0.43	▲ ▼ Удалить
4	2000	0.82	▲ ▼ Удалить
5	2500	1.41	▲ ▼ Удалить
6	3000	1.56	▲ ▼ Удалить

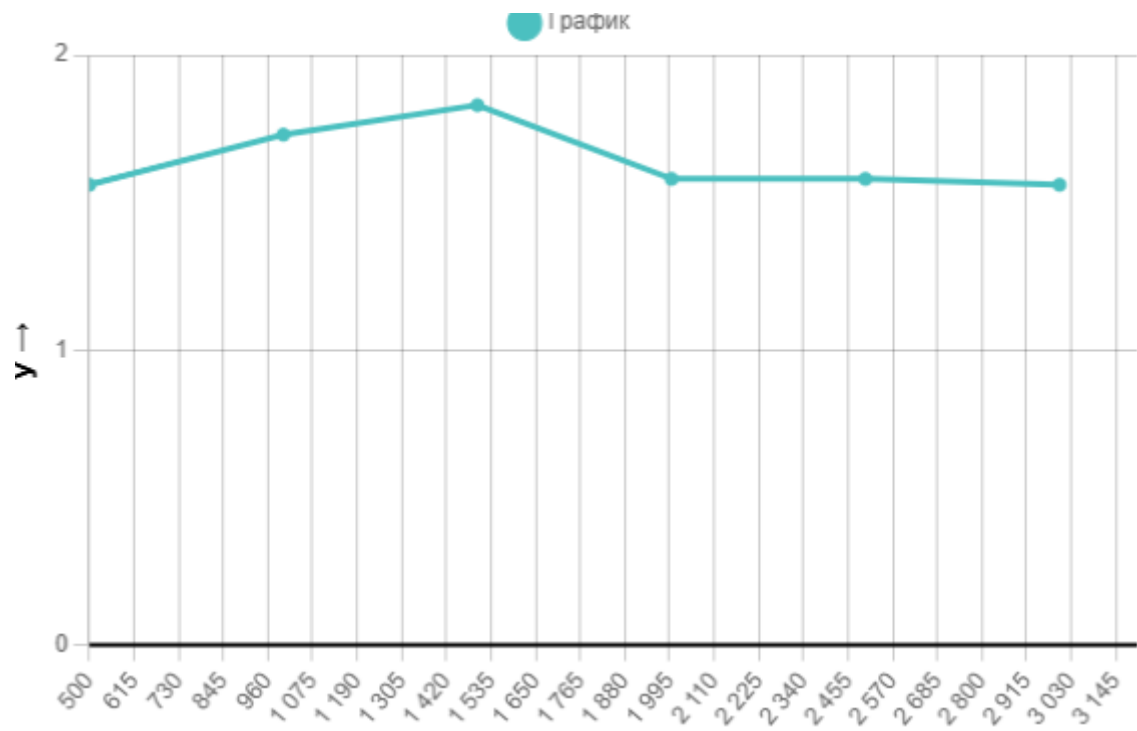
результат работы на точках разбитых на две части

алгоритмический метод



№	x	y	Действия
1	500	0.02	▲ ▼ Удалить
2	1000	0.08	▲ ▼ Удалить
3	1500	0.17	▲ ▼ Удалить
4	2000	0.82	▲ ▼ Удалить
5	2500	0.32	▲ ▼ Удалить
6	3000	0.56	▲ ▼ Удалить

работа ИИ



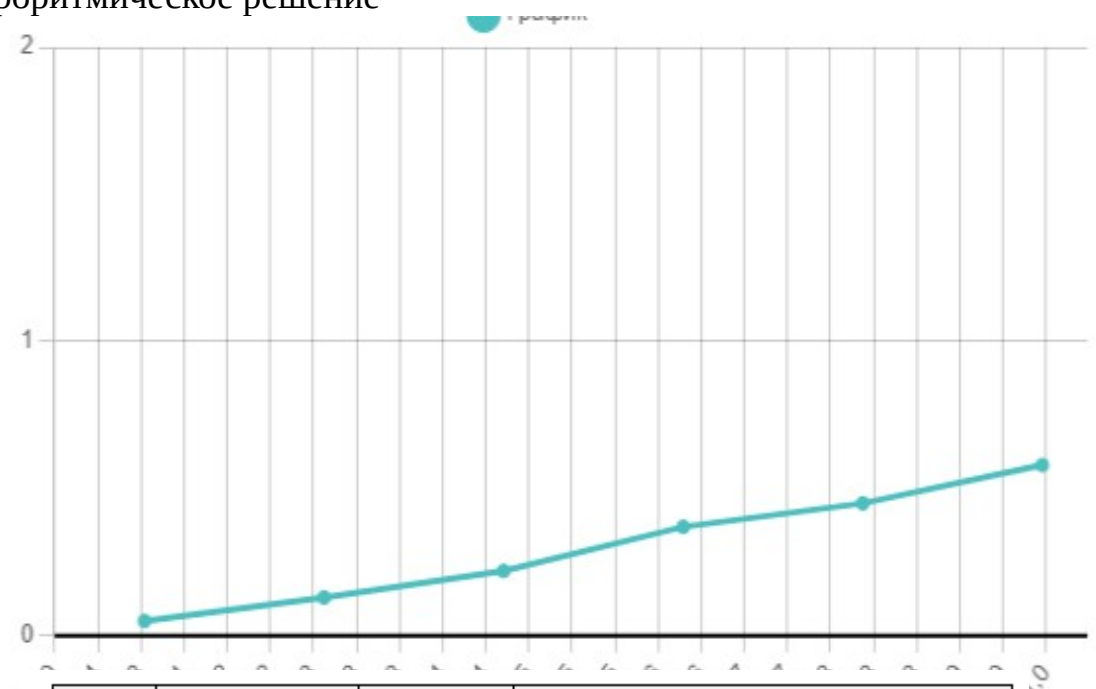
№	x	y	Действия
1	500	1.56	▲ ▼ Удалить
2	1000	1.73	▲ ▼ Удалить
3	1500	1.83	▲ ▼ Удалить
4	2000	1.58	▲ ▼ Удалить
5	2500	1.58	▲ ▼ Удалить
6	3000	1.56	▲ ▼ Удалить

к-средних

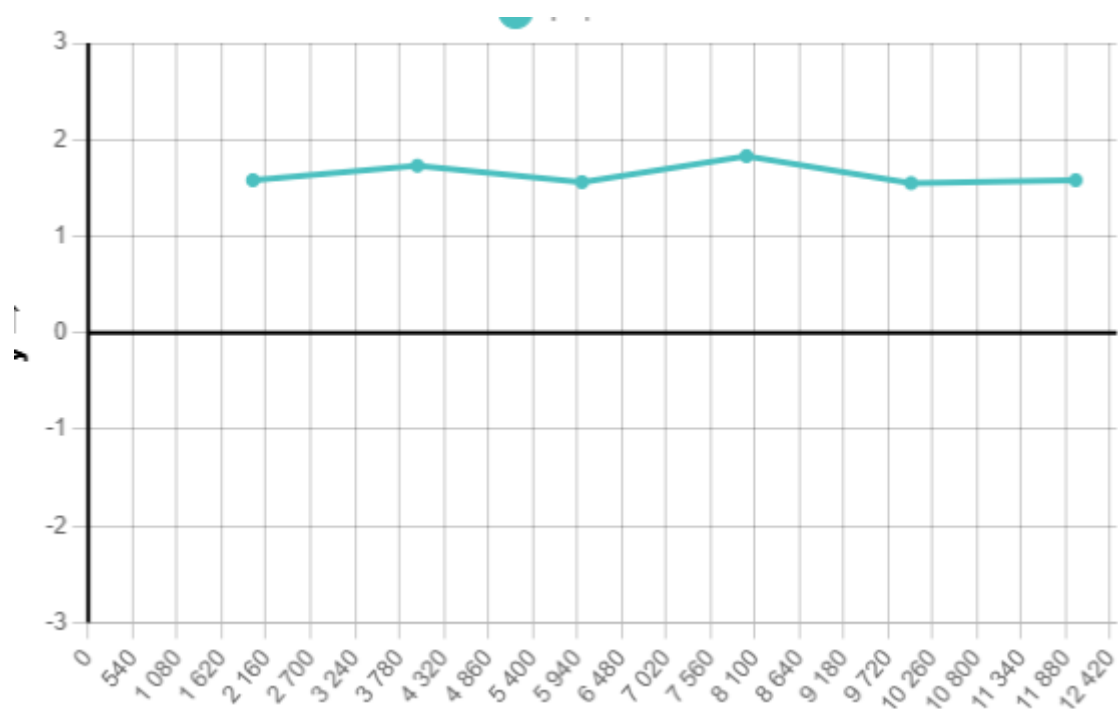


результат на регулярных данных

Алгоритмическое решение

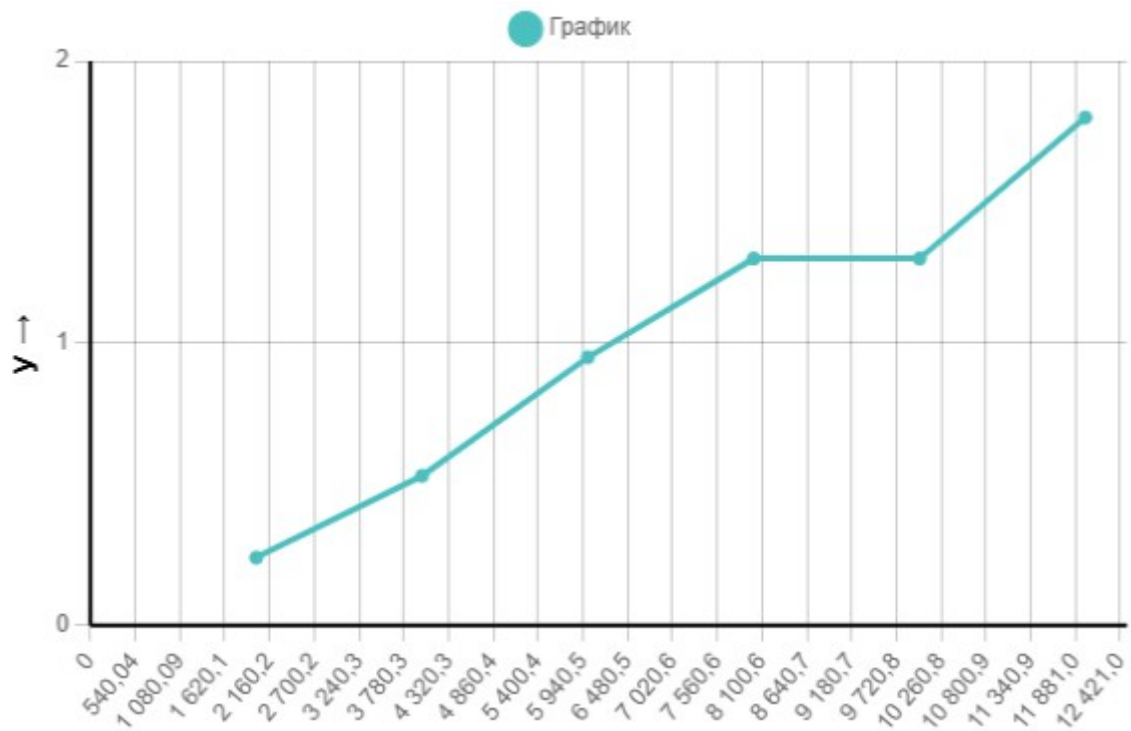


№	x	y	Действия
1	2000	0.05	▲ ▼ Удалить
2	4000	0.13	▲ ▼ Удалить
3	6000	0.22	▲ ▼ Удалить
4	8000	0.37	▲ ▼ Удалить
5	10000	0.45	▲ ▼ Удалить
6	12000	0.58	▲ ▼ Удалить



№	x	y	Действия
1	2000	1.58	▲ ▼ Удалить
2	4000	1.73	▲ ▼ Удалить
3	6000	1.56	▲ ▼ Удалить
4	8000	1.83	▲ ▼ Удалить
5	10000	1.55	▲ ▼ Удалить
6	12000	1.58	▲ ▼ Удалить

к-средних



№	x	y	Действия
1	2000	0.24	▲ ▼ Удалить
2	4000	0.53	▲ ▼ Удалить
3	6000	0.95	▲ ▼ Удалить
4	8000	1.3	▲ ▼ Удалить
5	10000	1.3	▲ ▼ Удалить
6	12000	1.8	▲ ▼ Удалить

Вывод: На небольшом количестве данных самым быстрым оказался алгоритмический метод к-средних, однако с увеличением примерно до 5000 тысяч ИИ выигрывает так как его время работы стабильно и не сильно меняется

с увеличением количества данных. Для больших данных ИИ более разумный выбор

КОД Алгоритмического и ИИ решения:

```
import pandas as pd
from sklearn.cluster import KMeans

import math
import time
import random as r
list_city = []
distance_matrix = []
line = ['0','0']
k_max = int(input("Введите кол-во кластеров "))
sum_x = 0
sum_y = 0
value_city = 0
index_city = 0
with open('city2.csv') as file: #читаем файл с городами
    while len(line) != 1 :
        line=file.readline().split(',')

        try:
            list_city.append((index_city,int(line[1]),int(line[2][0:-1]))) #при формировании списка
            # городов заменяем название города на индекс
            index_city +=1
        except:
            pass

    for first_name,fx,fy in list_city: # формируем матрицу смежности
        row_matrix = []
        sum_x += fx
        sum_y += fy
        value_city += 1
        for second_name,sx,sy in list_city:
            x = fx - sx
            y = fy - sy

            distance = (x**2)+(y**2)
            distance = round(math.sqrt(distance))
            row_matrix.append(distance)

        distance_matrix.append(row_matrix)
    # for row in distance_matrix: # вывод матрицы
    #     print(row)

def cluster_maker(list_city,distance_matrix,sum_x,sum_y,value_city,k_max): # самописный
    алгоритм создания кластеров

    mid_x = round(sum_x/value_city)
    mid_y = round(sum_y/value_city)
    distance_dict = {}
    cluster_dict = {}

    for name ,city x,city y in list_city: # вычисляем дистанции городов до средней точки
```

```

x = city_x - mid_x
y = city_y - mid_y
distance = round((x ** 2) + (y ** 2))
distance_dict[name] = distance

for i in range(1, k_max + 1): # находим самые удаленные точки
    key = max(distance_dict, key=distance_dict.get)
    cluster_dict[key] = []
    del distance_dict[key]

best_dict = {}
# каждая строка это определенный город ,в строке расстояния до других городов
# из строки мы выбираем расстояния до городов центроидов и ищем минимальное из
них ,то есть к кому ближе город
for row in distance_matrix:
    for i in cluster_dict.keys():
        best_dict[i] = row[i]

    key = min(best_dict, key=best_dict.get)
    name_city = distance_matrix.index(row)
    # if name_city not in cluster_dict.keys(): # если нужно чтобы центроид не был в
кластере
    cluster_dict[key].append(name_city)
print(f"ФИНИШ Кластеры: {cluster_dict}")

def sci_method(k_max): # создание кластеров через Scikit-learn

    # Задаём количество кластеров
    K = k_max

    # Загружаем данные из CSV-файла
    df = pd.read_csv('citySCI2.csv')

    # Извлекаем координаты для кластеризации
    X = df[['x', 'y']].values

    # Создаём модель KMeans и обучаем её
    kmeans = KMeans(n_clusters=K, random_state=42)
    kmeans.fit(X)

    # Получаем метки кластеров для каждого города
    df['cluster'] = kmeans.labels_

    # Выводим результат
    print(df)

def wcss_maker(): # метод локтя для k-means
    old_sum_distance = 1
    procent_distance = 0
    value_k = 0
    while procent_distance < 5:
        value_k += 1
        sum_distance =
k_means_madeself(k_max=value_k, list_city=list_city, distance_matrix=distance_matrix, wcss_m
aker_flag=True)
        procent_distance = ((old_sum_distance - sum_distance) / old_sum_distance) * 100
        old_sum_distance = sum_distance

    return value_k

def k_means_madeself(k_max, list_city, distance_matrix, wcss_maker_flag):
    cluster_dict = {} # словарь для кластеров

```

```

old_cluster_dict = {} # сохраняем старый набор кластеров
sum_distance = 0 # сумма квадратов расстояния для метода локтя
for i in range(1, k_max+1): # выбираем случайные точки
    key = r.randint(0, len(list_city)-1)
    cluster_dict[key] = []
if wcss_maker_flag == False:
    print(f'первый {cluster_dict}\n')
while old_cluster_dict.keys() != cluster_dict.keys():
    best_dict = {}

    #определение городов в кластеры
    for row in distance_matrix:
        for i in cluster_dict.keys():
            best_dict[i] = row[i]

    key = min(best_dict, key=best_dict.get)
    name_city = distance_matrix.index(row)
    # if name_city not in cluster_dict.keys(): # если нужно чтобы центроиды не были в
кластере
    cluster_dict[key].append(name_city)

    sum_x=0
    sum_y=0
    sum_distance = 0
    list_next_centroid = []
    # для каждого кластера находим среднюю точку и ближайший к ней город
    for key in cluster_dict:
        for city in cluster_dict[key]:
            sum_x += list_city[city][1]
            sum_y += list_city[city][2]
        mid_x = round(sum_x / value_city)
        mid_y = round(sum_y / value_city)
        min_distance = None
        next_centroid_cluster = 0
        for city in cluster_dict[key]: #поиск наиболее приближенного города к средней
координате кластера
            x = list_city[city][1] - mid_x
            y = list_city[city][2] - mid_y
            distance = round((x ** 2) + (y ** 2))
            if wcss_maker_flag:
                sum_distance += distance
            if min_distance is None or distance < min_distance:
                min_distance = distance
                next_centroid_cluster=city
        sum_x=0
        sum_y=0
        list_next_centroid.append(next_centroid_cluster)
    old_cluster_dict = cluster_dict.copy() # сохраняем старый набор кластеров
    cluster_dict = {}
    for key in list_next_centroid: # создаем новый набор кластеров
        cluster_dict[key] = []
    if wcss_maker_flag == False:
        print(f'новый {cluster_dict}')
        print(f'старый {old_cluster_dict}\n')
    else:
        return sum_distance

print("СОВПАДЕНИЕ! конец")

```

```
start_time = time.time()
cluster_maker(list_city=list_city,distance_matrix=distance_matrix,sum_x=sum_x,sum_y=sum_y,value_city=value_city,k_max=k_max)
end_time = time.time()
print(f"Время выполнения собственной функции {end_time-start_time:.4f}")
start_time = time.time()
sci_method(k_max=k_max)
end_time = time.time()
print(f"Время выполнения через SCIKIT {end_time-start_time:.4f}")
print("-----")
elbow_k_max = wcss_maker()
start_time = time.time()
k_means_maker(k_max=elbow_k_max,list_city=list_city,distance_matrix=distance_matrix,wcss_maker_flag=False)
end_time = time.time()
print(f"Время выполнения через K-Means {end_time-start_time:.4f}")
```